

Algorithms

give more importance on code

prev sem covered topic won't be covered in this sem.

• graph advance part

dp → after mid

Algorithms: finite set of instructions (~~that~~ would take finite time)

- | | | |
|------------------|--|--|
| 1. input/s |  | algo can't be ambiguous |
| 2. output/s | | ambiguous. (a/o ; adding 6 or 7 to x) |
| 3. Definiteness | | |
| 4. Finiteness | | an algo can take a year to halt. But it should have a feasible and finite time |
| 5. effectiveness | | code complexity. |

Areas of study

- How to devise an algorithm.
- How to validate an algorithm.
- How to analyze an algorithm.
- How to test an

Devise an algorithm $\rightarrow$ Probable solⁿ

(theory) validate " " $\rightarrow$ check the correctness

analyze " " $\rightarrow$ check if the algo is efficient (time & memory)

(practical) test " " $\rightarrow$ ~~compare~~

conventions of writing Pseudocode

Comments : "//" use this to write comment

Blocks : { }

Identifiers : variable, function names.
defining the type is not mandatory.

compound variable (struct) $\rightarrow$ node = record {

#

val;
node *next;
}

Assignment: in code we use '=' but here ':='

Logical operators: =, <, $\leq$, >, $\geq$, $\neq$

Multidimensional Arrays: for accessing $A[i, j, k]$
In code $A[i][j][k]$

Loops : for loop $\rightarrow$

```
For val := value 10 to value 2 step step do  
{  
    i = 0  
    i < 10  
    i = i + 2  
}
```

	<u>do while</u>
<pre>while <condition> do { ... }</pre>	<pre>Repeat : until (condition)</pre>

Conditionals :

```
If condition then  
{  
    ...  
}  
else {  
    ...  
}
```

```
If condition then  
{ ... }  
else If condition Then  
{ ... }
```

Case {

: condition 1 : { statements }

: condition 2 : { " }

: else : { }

}

Procedure / Function :

Algorithm function Name (Parameters) {

return;

}

read input → read / READ / Read

Given a set of numbers A , and a number x , find if the number x exist in the set A .

$A = \{a_1, a_2, a_3, \dots, a_n\}$ $\left\{ \begin{array}{l} \text{sorted} \rightarrow \text{binary search} \\ \max(A) < 100, \end{array} \right.$

```
Algorithm Find ( $A, x$ ) {  
    length := A.length;  
    For  $i := 0$  to length step 1 do {  
        If  $A[i] = x$  then  
            return True;  
    }  
    return False;  
}
```

* ~~peak~~ finding dimensional array
binary search

Algorithms

recursive Algorithm

* almost same as recursive function.
can be defined in 2 ways

the algorithms that calls itself directly or indirectly.

Direct Recursion:

$$f(n) = f(n-1) + f(n-2)$$

Indirect Recursion:

```
func1() {  
    func2();  
}
```

```
func2() {  
    func1();  
}
```

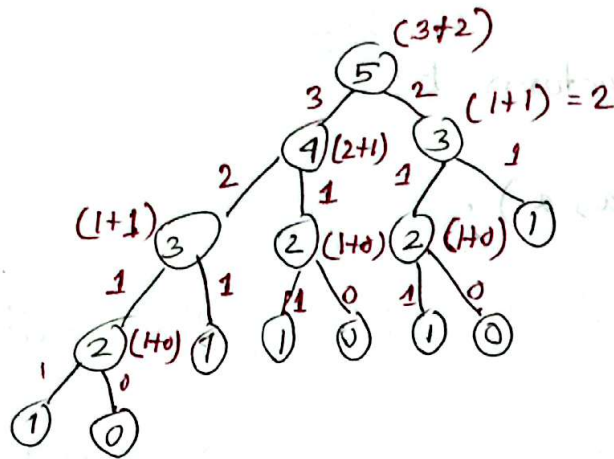
* It can be used anywhere ~~in~~ in place of loop.

① Fibonacci series

$$n_i = n_{i-1} + n_{i-2}$$

Algorithm nthFib(n) {
 if (n=0) return 0; if (n=1) return 1;
 nthf := nthFib(n-1) + nthFib(n-2);
 return nthf;
}

n = 5



Example :

Binomial coefficient calculation :

n=5 r=3

$${}^nC_r = {}^{n-1}C_{r-1} + {}^{n-1}C_r$$

if n = 1 return 1

if r = 1 return n

②

a b c d e

5 টা থেকে 3 টা select করে পাওয়া যায়,

a কে fix করলে বাকি 4 টা থেকে 2 টা

নিতে হবে 4C_2

fix না করলে a না বাকি

দিলে 4C_3

$${}^5C_3 = {}^4C_2 + {}^4C_3$$

③

GCD

12, 9

$$\begin{array}{r} \overline{a} \quad \overline{b} \\ 9 \mid 12 \mid 1 \\ \underline{9} \\ 3 \mid 2 \mid 3 \\ \underline{9} \\ 0 \end{array}$$

gcd

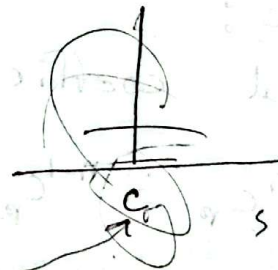
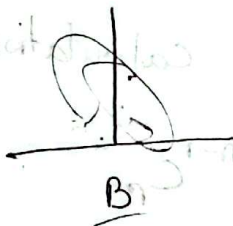
→ euclidean method.

```
gcd(a, b) {  
    if (a = 0) return b;  
    return gcd(b % a, a);  
}
```

④

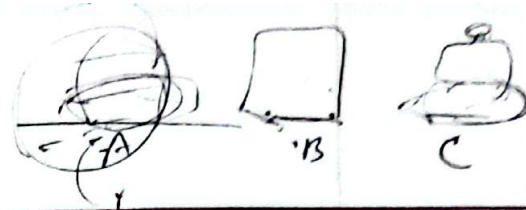
Tower of Hanoi

n=4



n = num of disk

num of disk
to move



Tower (n, A, B, C) {

if (n == 0) return;

Tower (n-1, A, C, B)

write (Move top most disk from A to B);

Tower (n-1, C, B, A)

}

⑤

Generating all possible combination:

perm (Arr, k, n) {

Arr[k],

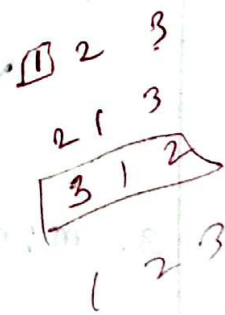
For i := k to n {

swap (Arr[k], Arr[i])

perm (Arr, k+1, n)

}

swap (Arr[k], Arr[k]);



Algorithms

Generate All possible ordering / permutation

$A = \{a, b, c, d, e, f\}$

Algorithm $m(A, k, n)$

$\{$
if $(k == n-1)$ write (A) return;
for $(i := k \text{ to } n-1)$ do
 $\{$ swap $(A[k], A[i]);$
 Algorithm $m(A, k+1, n);$
 swap $(A[k], A[i]);$
 $\}$
 $\}$

$b | a \longrightarrow \begin{pmatrix} a & b & c \\ a & c & b \end{pmatrix}$

$\begin{pmatrix} b & a & c \\ b & c & a \end{pmatrix}$

algo $(A, 0, n)$
but I don't want
to change the
first element
so, algo $(A, 1, n)$

Performance Analysis

space complexity: memory required from starting
the execution to the completion
of the program.

constant space

Algo

Algorithm abc (a, b, c) {

return a + b + b * c + (a + b - c) / (a + b) + 4.0;

}

Q

Algorithm sum (A, n) {

s := 0.0; → 1

for i := 0 to n-1 do {

s := s + A[i]; → 1

}

return s; → 1

}

t.c. → $O(2n+3)$

$O(n)$ → arr size

Algo RSum (A, n) {

if (n < 0) return 0.0;

return RSum(A, n-1) + A[n-1];

↓ stack memory

$O(n)$

why n+1 because it needs to check the breaking condition.

space complexity:

1. Fixed memory components

in sum s, i etc are fixed.

2. variable memory components
in Sum Array is the variable memory
components.

$$S(P) = c + S_p(\text{Problem instance})$$

Problem Algo fixed variable memory components

Time Complexity :

$$T(P) = \underbrace{C(P)}_{\text{this depends on the language and compiler}} + \underbrace{t_p(\text{problem instance})}_{\text{only consider this}}$$

$$t_p(n) = C_a * \text{ADD}(n) + C_s * \text{SUB}(n) + C_m * \text{MUL}(n)$$

$$+ C_d * \text{DIV}(n) + \dots$$

Algo Rsum(A, n)

{

if (n < 0)

return 0.0;

return (Rsum(A, n-1) + A[n-1]);

}

	n < 0	n > 0
①	0	0
②	1	1
③	1	0
④	0	1+x
	2	2+x

↑
t_p Rsum(n-1)

∴ O(n)

t_p Rsum(n) → 2 + 2 + t_p Rsum(n-1);

frequency

Algo Add (A, B, C, n, m) {

for (i := 0 to n do {

for (j := 0 to m do {

C[i, j] = A[i, j] + B[i, j];

}

}

}

→ 0

→ 1

→ 1

→ 1

→ n+1

→ n × (m+1)

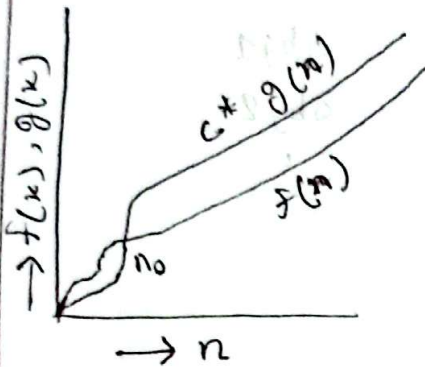
→ n × m

$2mn + 2m + 1$

Algorithm

Asymptotic Notation O, Θ, Ω

O



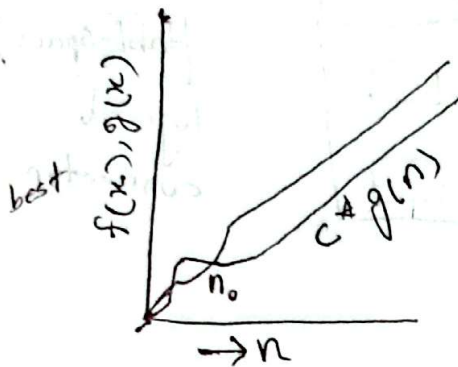
$$f(n) \leq c * g(n)$$

$$f(n) \rightarrow O(g(n))$$

$$n \geq n_0$$

$f(n)$ is upper-bounded by $g(n)$

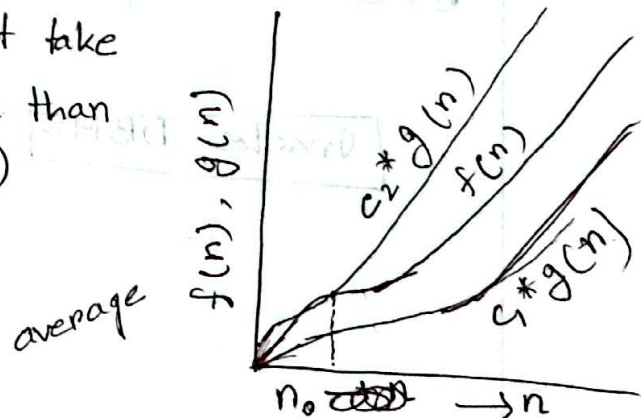
Ω



$$f(n) \geq c * g(n)$$

Algo won't take less time than $\Omega(g(n))$

Θ



bounded from both sides

$$c_1 * g(n) \leq f(n) \leq c_2 * g(n)$$

example : $f(n) = 2n + 3$, $g(n) = 3n$

$n=1$	5	3
$n=2$	7	6
$n=3$	9	9
$n=4$	11	12

$$n_0 = 3 \quad n > 3 \quad c = 1$$

$$\therefore f(n) \leq g(n) \quad \therefore f(n) \rightarrow O(g(n))$$

$$O(3n)$$

$$g(n) = n^2 \quad n_0 = 3$$

$$f(n) \rightarrow O(n^2)$$

$$f(n) \rightarrow O(n^3)$$

for calculating complexity
algo won't take more
than $O(3n)$ time.

$$n^2 + 5 \leq c_2 n^2$$

$$5 \leq c_2 n^2 - n^2$$

$$5 \leq n^2 (c_2 - 1)$$

$$\frac{5}{n^2} \leq c_2 - 1$$

$$\frac{5}{n^2} + 1 \leq c_2$$

$$n_0 = 3 \rightarrow c_2 = 2$$

~~Ex~~

example

$$f(n) = n^2 + 5$$

$$g(n) = n^2$$

$$f(n) \geq g(n)$$

$$c_1 = 1$$

$$c_2 = 2$$

$$f(n) \leq 2g(n)$$

$$n_0 = 3$$

$$f(n) \rightarrow \Theta(g(n)) = \Theta(n^2)$$

Divide and Conquer

- 1) Divide the problem into smaller subproblems.
- 2) solve the smaller subproblem.
- 3) Combine the results of sub problems.

Algorithm DAndC(P) {

if small(P) then S(P);

else {

Divide the problem P into subproblems

$P_1, P_2, P_3 \dots P_k \quad k \geq 1$

}

call the DAndC over all these $P_1, P_2 \dots P_k$.

return combine(DAndC(P_1), DAndC(P_2), ..., DAndC(P_k)).

}

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \text{ small enough} \\ T(n_1) + T(n_2) + \dots + T(n_k) & \text{otherwise} \end{cases}$$

Binary Search

Algorithm BS (Arr, left, right) {

if small (left, right) then return S (left, right)

else {

mid := (left + right) / 2

if (check (Arr, mid)) then

return BS (Arr, left, mid);

else

return BS (Arr, mid+1, r);

}

$$T(n) = \begin{cases} T(1) & n \text{ small enough} \rightarrow \Theta(1) \\ T(n/2) & \end{cases}$$

for BS

$$T(n/2) + f(n) \rightarrow \Theta(1)$$

complexity of
Divide part

$$T(n) = T(n/2)$$

$$T(n/2) = T(n/4)$$

$$T(n/4) = T(n/8)$$

for all the recurrence

call $f(n) \log_2 n$

$$T(n/2^i) = T(n/2^{i+1})$$

$$\frac{\Theta(1) \log_2 n + \Theta(g(n))}{\Theta(1)} \rightarrow$$

$$T(1) = T(n/2^i)$$

$$\Rightarrow \log_2 n$$

$$(\text{Depth})^i = \log_2 n$$

Algorithm FindMaxMin (arr, left, right, max, min) {

if (right - left == 1) then

if ($A[l] < A[r]$) then

{

max := $A[r]$;

min := $A[l]$;

}

else {

max := $A[l]$;

min := $A[r]$; }

}

1 2 4 8

else {

Divide - $\text{mid} := (l+r)/2$;

conquer

FindMaxMin (Arr, l, mid, max₁, min₁);

FindMaxMin (Arr, mid+1, r, max₂, min₂);

combine

if (max₁ < max₂) then max := max₂;

else max := max₁;

if (min₁ < min₂) then min := min₁;

else min := min₂;

}

}

$$T(n) = \begin{cases} T(1) \rightarrow \Theta(1) & n \text{ small enough} \\ 2T(n/2) + f(n) \rightarrow \Theta(1) \end{cases}$$

$$\Theta(1) \times 2^{\log_2 n} \times \log_2 n + \Theta(1)$$

$$O(n \log_2 n)$$

$n/2$

$$T(n) = 2T(n/2)$$

$$= 4T(n/4)$$

$$= 8T(n/8)$$

$$= 2^i T(n/2^i)$$

$$i = \log_2 n$$

R O D O

for merge sort

$T(1)$

$$T(n) = 2T(n/2) + f(n) \rightarrow \Theta(n)$$

DBMS - II

* why do we need (tablespace?)

↓
lowest level of control in terms
of physical storage -

* read only

* offline

* online

Algorithms

Peak Finding

Problem P: Given an array of n numbers find any peak $A[i]$ is a ^{peak} pick if

$$A[i] \geq A[i-1] \text{ where } i > 1 \text{ and}$$

$$A[i] \geq A[i+1] \text{ where } i < n$$

straight forward approach:

Algorithm 1D Peak (A, n) {

for $i := 1$ to n do {
 flagLeft := false; flagRight := false
 if $i > 1$ and $A[i] \geq A[i-1]$ then

 flagLeft := true;

 if ($i < n$ and $A[i] \geq A[i+1]$) then

 flagRight := true;

if flagLeft And flagRight then

 return $A[i]$;

}

```

if  $n=1$  or  $A[1] \geq A[2]$  then return  $A[1]$ 
if  $A[n] \geq A[n-1]$  then return  $A[n]$ ;
}

```

Divide and conquer approach

Algorithm OneDPeak (A, l, r) {

if ($l = r$) return $A[l]$;

$m := (l+r)/2$;

else if $m > 1$ and $A[m] \leq A[m-1]$ then

return OneDPeak ($A, l, m-1$);

else if $m > 1$ and $m < n$ and $A[m] \geq A[m-1]$ and

$A[m] \geq A[m+1]$

return $A[m]$;

else

return OneDPeak ($A, m+1, r$);

}

$$T(n) = \begin{cases} T(1) & A[m] \text{ is a peak} \\ T(n/2) + \Theta(1) & A[m] \text{ is not a peak} \end{cases}$$

$$= T(n/4) = T(n/2^2)$$

$$= T(n/8) = T(n/2^3)$$

$$= T(n/2^i)$$

$$i = \log_2 n$$

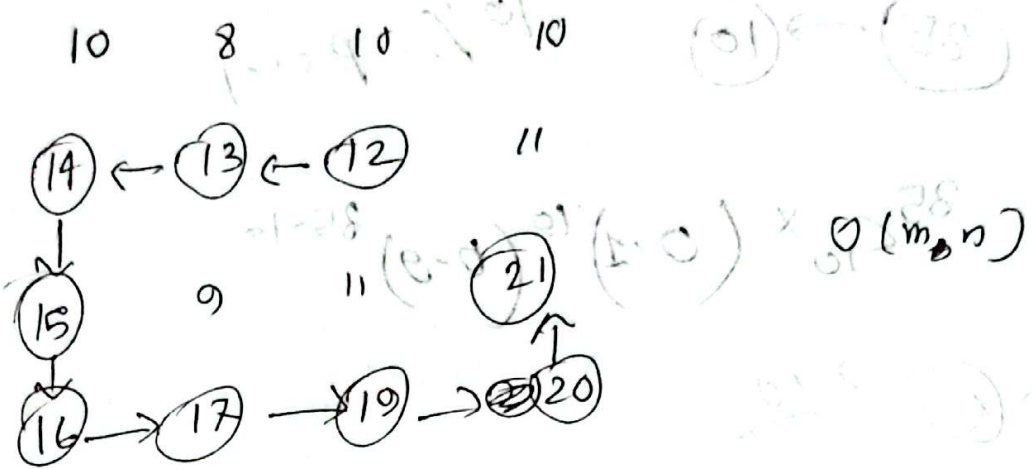
* Given a 2D array of size $m \times n$, find a peak. $A[i, j]$ is a peak if:

$$A[i, j] \geq A[i-1, j], \quad i > 1$$

$$A[i, j] \geq A[i+1, j], \quad i < m$$

$$A[i, j] \geq A[i, j-1], \quad j > 1$$

$$A[i, j] \geq A[i, j+1], \quad j < n$$



D. C.

$$T(n, m) = \begin{cases} \theta(n) \\ T(n, m/2) + \theta(n) \rightarrow \theta(n) \end{cases}$$

$$= T(n, m/2^2) + \theta(n)$$

$$T(n, m/2^3) + \theta(n)$$

⋮

$$(n, m/2^i) + \theta(n)$$

$$i = \log_2 m$$

Algorithm

Complexity Analysis of Divide & Conquer

Solving Recurrence, $T(n) = a T(n/b) + f(n)$

Annotations:

- a : number of sub-problems
- n/b : size of each sub problem
- $f(n)$: cost of dividing
- b : factor for divide the problem

Binary Search,

$$T(n) = T(n/2) + \theta(1)$$

Annotations:

- $n/2$: working with the half part
- $\theta(1)$: dividing $\text{mid} = (\text{left} + \text{right})/2$

3 ways of solving these recurrence:

- ① Substitution Method.
- ② Recurrence Tree Method.
- ③ Master Method.

Substitution Method: (is for proving that the complexity is right)

- * guess the complexity mathematical
- * Use then Induction to prove that the guess is correct.

Assume $T(n) \leq c \log n$ where $n \geq n_0$ and $c > 1$

$$T(n) = T(n/2) + \theta(1) \quad \left| \begin{array}{l} \text{let,} \\ \theta(1) \rightarrow c_2 (\text{constant}) \end{array} \right.$$

$$= c \log n/2 + c_2$$

$$= c \log n - \underbrace{c \log 2}_{\text{constant term}} + c_2$$

$$= c \log n - c + c_2$$

$$= c \log n - (c - c_2)$$

$$T(n) \leq c \log n \quad \text{when } (c \geq c_2)$$

Merge Sort:

$$T(n) = 2T(n/2) + c_2 n \rightarrow \text{in merging part}$$

(height of)

$$O(n) \rightarrow c_2 n$$

Assume $T(n) \leq Cn \log n$ where $n \geq n_0$ and $C \geq 1$

$$T(n) = 2T(n/2) + c_2 n$$

$$= 2C \frac{n}{2} \log \frac{n}{2} + c_2 n$$

$$= 2Cn \log n - 2Cn \log 2 + c_2 n$$

$$= n(2C \log n - 2C \log 2 + c_2)$$

$$= n \{ 2C \log n - (2C \log 2 - c_2) \}$$

$$\leq n \underbrace{2C \log n}_{\text{constant}}$$

$$T(n) \leq Cn \log n$$

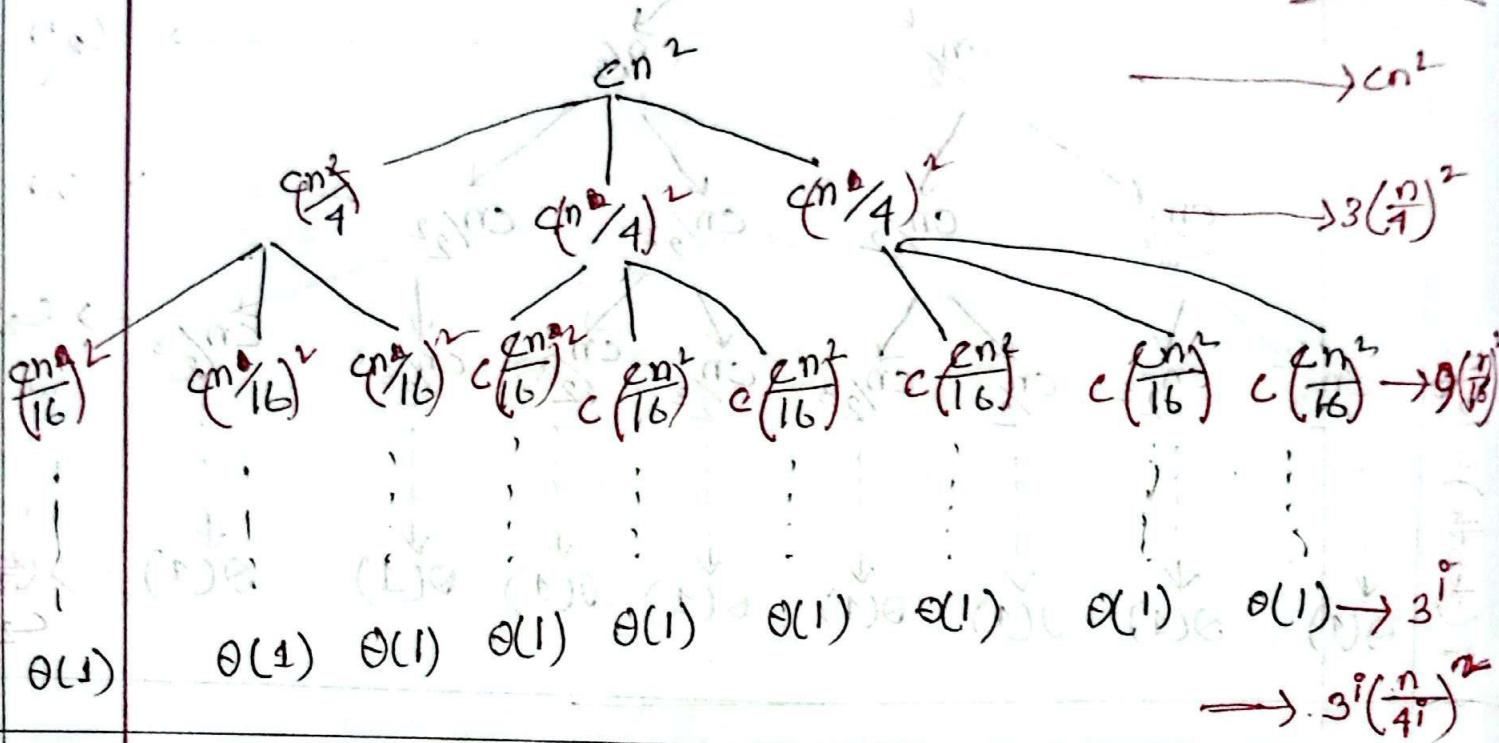
Recurrence Tree :

$$T(n) = 2T(n/2) + c_2 n$$

$$T(n) = 3T\left(\frac{n}{4}\right) + \theta(n^2)$$

number of Problems

$n/4^i = 1; i = \log_4 n$



$$T(n) = cn^2 + 3c\left(\frac{n}{4}\right)^2 + 3^2c\left(\frac{n}{4^2}\right)^2 + \dots + 3^i c\left(\frac{n}{4^i}\right)^2$$

$$= cn^2 \left(1 + \frac{3}{4^2} + \frac{3^2}{4^4} + \dots + \frac{3^i}{4^{2i}}\right) + c_2 3^{\log_4 n}$$

$$= cn^2 \left(1 + \frac{3}{4^2} + \frac{3^2}{4^4} + \dots + \frac{3^i}{4^{2i}}\right) + c_2 3^{\log_4 n}$$

$$= cn^2 \sum_{i=0}^{\log_4 n} \left(\frac{3}{16}\right)^i + c_2 3^{\log_4 n}$$

$$T(n) = cn^2 \sum_{i=0}^{\log_4 n} \left(\frac{3}{16}\right)^i + c_2 3^{\log_4 n}$$

$$\leq cn^2 \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i + c_2 3^{\log_4 n}$$

$$\leq cn^2 \frac{1}{1 - \frac{3}{16}} + c_2 3^{\log_4 n}$$

$$\leq \frac{16cn^2}{13} + c_2 3^{\log_4 n}$$

$$\leq c_3 n^2 + c_2 n^{\log_4 3}$$

where $c_3 = \frac{16c}{13}$

$$\leq c_3 n^2 + c_2 n^{0.8}$$

$$O(n^2 + n^{0.8})$$

$$= O(n^2)$$

Formula:

If $r < 1$,

$$r + r^2 + r^3 + \dots$$

$$= \left(\frac{1}{1-r}\right)$$

$$a^{\log_b c} = c^{\log_b a}$$

Algorithm

Master, Method

$$T(n) = aT(n/b) + f(n)$$

Case 1:

if $f(n) = O(n^{\log_b a - \epsilon})$ where $\epsilon > 0$, then

$$T(n) = O(n^{\log_b a})$$

in the tree, it represents leaf level complexity

Case 2:

If $f(n) = \Theta(\underbrace{n^{\log_b a}}_{\substack{\text{in the tree, it} \\ \text{represents leaf level complexity}}} \log^k n)$ for $k \geq 0$, then

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

Case 3:

If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for $\epsilon > 0$ and

$af(n/b) \leq cf(n)$ where $c > 1$ and n sufficiently

larger then $T(n) = \Theta(f(n))$

Example:

$$T(n) = 2T(n/2) + n$$

$$\text{So, } n^{\log_2 2} \log^0 n \Rightarrow n^1 \cdot 1 \Rightarrow n \Rightarrow f(n)$$

Falls under case 2

$$\text{so } T(n) = n^{\log_2 2} \log^{0+1} n \Rightarrow n \log n$$

example 2:

$$T(n) = 3T(n/4) + n^2$$

$$a = 3, b = 4$$

$$\text{so, } n^{\log_4 3} \Rightarrow n^{\log_4 3 + \epsilon} \Rightarrow n^{\log_4 16} \Rightarrow n^2$$

$$f(n/4) = \frac{n^2}{16} \quad f(n) = n^2$$

$$3f(n/4) = \frac{3n^2}{16}$$

$$\leq \left(\frac{1}{2}\right)n^2 \quad c < 1$$

so, falls under case 3:

$$\text{therefore } T(n) = \Theta(f(n)) \Rightarrow \Theta(n^2)$$

Example 3:

$$T(n) = 2T(n/2) + \Theta(1)$$

$$; a = 2, b = 2$$

$$n^{\log_2 2}$$

$$\Rightarrow n^{\log_2 2 + \epsilon} \Rightarrow n^{\log_2 4} \Rightarrow n^2$$

$$n^{\log_2 1}$$

$$= n^0 = 1 = \Theta(1)$$

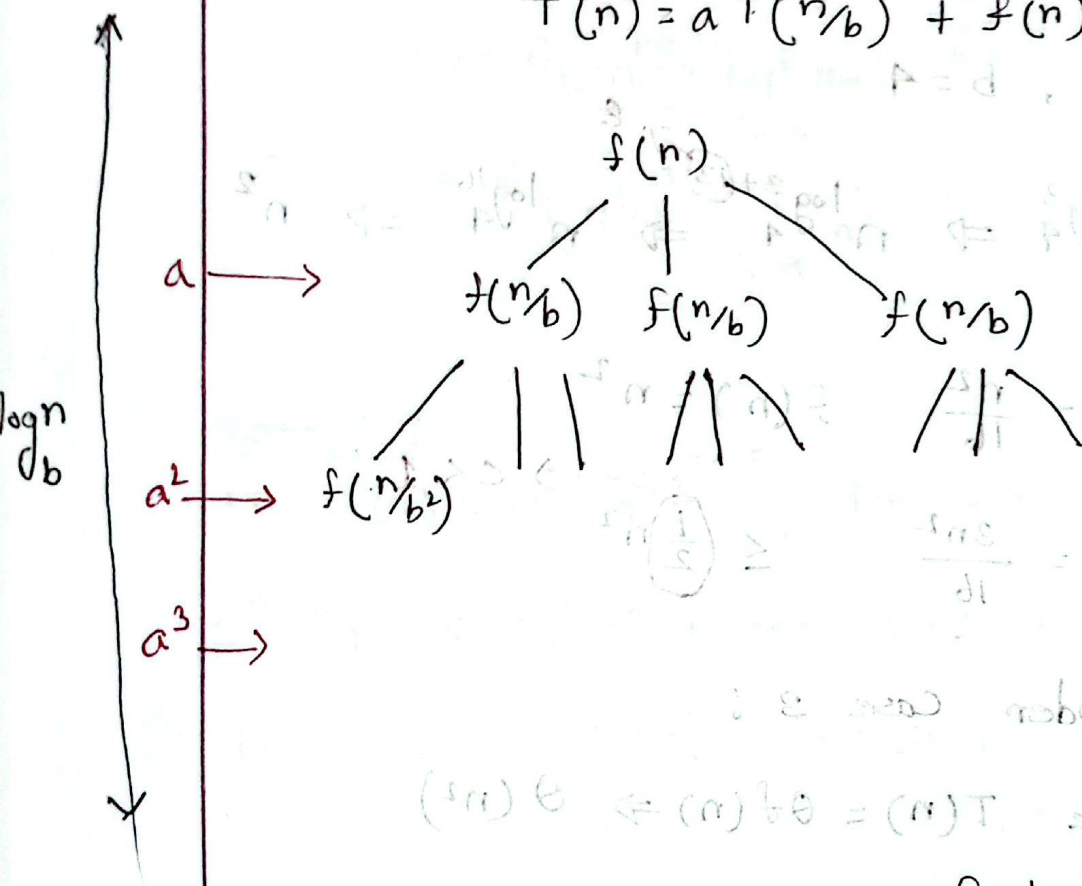
[level base] $(\log n)$

falls under case 1;

$$\text{So, } T(n) = O(n^{\log_2 2})$$

For the cases what can we get using recurrence tree

$$T(n) = aT(n/b) + f(n)$$



in case of depth i .

$$f\left(\frac{n}{b^i}\right)$$

$f(1) \rightarrow \frac{n}{b^i} \approx 1$ (ii) $i = \log_b n$

in the leaf nodes there will be a^i subproblems.
 $a^i \Rightarrow a^{\log_b n} \Rightarrow n^{\log_b a} \rightarrow$ these number of $O(1)$ problem
 $\therefore \Rightarrow O(n^{\log_b a})$ [leaf level]

each level complexity $\rightarrow f(n)$

To get to the i th depth total complexity

$$i = \log_b^n * f(n) ?$$

$$\therefore \text{Total complexity} = O(n^{\log_b a}) + \log_b^n * f(n)$$

In case 2, $n^{\log_b a}$ dominates the breaking part.
And represents that each level needs same amount of complexity.

Algot Algorithm

Greedy Algorithm

Feasible Solution :

subset of the problem that satisfy certain conditions fixed by the problem.

Objective Function :

Tries to maximize or minimize a function's value based on the feasible solutions.

Optimal solution :

Among all the feasible solution those solutions that maximize or minimize the objective function.

subset Paradigm :

ordering paradigm :

OS $\leftarrow (0, 0, 1)$

OS $\leftarrow (1, 0, 0)$

OS $\leftarrow (0, 1, 0)$

Fractional knapsack

n objects, A knapsack bag with capacity m
weight of n objects $w = \{w_1, w_2, w_3, w_4, \dots, w_n\}$

profit of n objects $p = \{p_1, p_2, p_3, \dots, p_n\}$

Objectives maximize $\sum_{i \in \text{objects}} x_i * p_i$ $\left\{ \begin{array}{l} 0 \leq x_i \leq 1 \end{array} \right.$

feasibility $\rightarrow \sum_{i \in \text{objects}} x_i * w_i \leq m$

$n = 3$ $m = 20$ $p = \{25, 24, 15\}$

$w = 9, p = 12.5$ $w = 5, p = 8$ $w = 2.5, p = 3.75$ $w = \{18, 15, 10\}$
 $\rightarrow (\frac{1}{2}, \frac{1}{3}, \frac{1}{4}) \rightarrow \frac{\text{total weight}}{16.5} \rightarrow \frac{\text{total profit}}{24.25}$

$\rightarrow (1, \frac{2}{15}, 0) \rightarrow 20 \rightarrow 28.2$

$\rightarrow (0, \frac{2}{3}, 1) \rightarrow 20 \rightarrow 31$

$\rightarrow (0, 1, \frac{1}{2}) \rightarrow 20 \rightarrow 31.5$

subset
paradigm

SRS

Backlog Refinement

Scope creep $\rightarrow$ ~~less~~ ^{more} ~~expand~~ ^{expand}

Algorithm

$n = 4$

$$P = \{100, 10, 15, 17\} \quad d = \{2, 1, 2, 1\}$$

n Jobs with deadline for each d_i and profit P_i
each jobs needs 1 unit of time to finish
what is the maximum profit can be gained
without overlapping any jobs.

Jobs	seq	profit
$\{1, 2\}$	2, 1	110
$\{1, 4\}$	4, 1	127
$\{1, 3\}$	1, 3 or 3, 1	15
$\vdots$	$\vdots$	$\vdots$

Optimal $\{1, 4\}$ with sequence $\{4, 1\}$ profit 127



going to take the most profit at the end of the deadline $\rightarrow$ index. then take profit decrease decreasing order and put them before the highest profit.

Algorithm JobSchedule(J, d, P, n) {

sort(J) descending based on profit
profit := 0;

for $i = 1$ to n do

for $s := d[i]$ to 1 do

if slot $[s]$ is empty then

assign slot $[s]$ to job $J[i]$

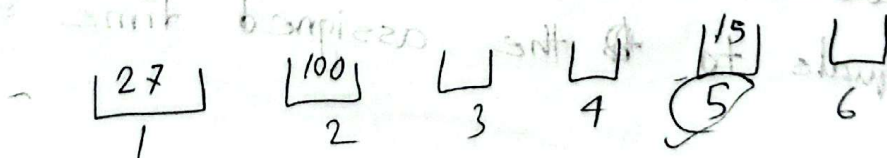
profit := profit + $P[i]$;

break;

return profit;

}

$P = \{100, 27, 15\}$, $d = \{2, 1, 5\}$



Theorem:

J is a set of k jobs and $\sigma = i_1, i_2, i_3, i_4, \dots, i_k$ is a permutation of 1 to k ,

such that $d_{i_1} \leq d_{i_2} \leq \dots < d_{i_k}$. Now J is a feasible solution iff jobs in J can be processed with the sequence σ without any overlaps.

$$d = \{ 7 \quad 8 \quad 9 \quad 12 \quad 11 \quad 10 \}$$

$$\text{jobs} = \{ 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \}$$

$$7 \leq 8 \leq 9 \leq 10 \leq 11 \leq 12$$

$$\sigma = \{ 1 \quad 2 \quad 3 \quad (6) \quad 5 \quad 4 \}$$

$$q = 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6$$

take it as a time slot

Proof

J is feasible for sequence $\sigma' = r_1, r_2, \dots, r_k$ if $d_{r_q} \geq q$ where $1 < q < k$.

(the deadline of the job should be greater than or equals to the assigned time slot)

Assume $\sigma' \neq \sigma$

The least index a where $i_a \neq p_a$
 $p_b \neq i_a, b > a$

$$\begin{cases} \sigma' = p_1, p_2, p_3 \dots p_b \dots p_k \\ \sigma = i_1, i_2, i_3 \dots i_c \dots i_k \end{cases}$$

Here the value of $a = 3$

~~The least index~~

$$\begin{array}{ccc} dr_3 > dr_b & , & c > 3 \\ \downarrow & \downarrow & \\ d_{ic} & d_{i_3} & \end{array}$$

if, $dr_a \geq dr_b$

Swap (p_b, p_a)

So, we can say,

$$dr_b \geq dr_a$$

$$\begin{array}{ll} as, & dr_b \geq b \quad b > a \\ & dr_a \geq a \quad dr_b \neq dr_a \\ & \Rightarrow \text{not always.} \end{array}$$

SRS

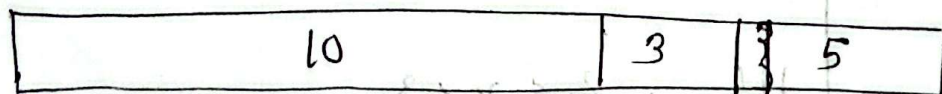
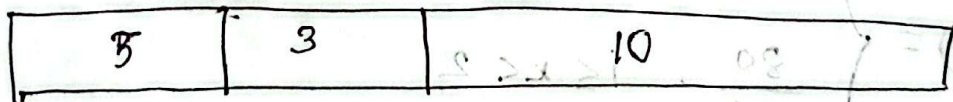
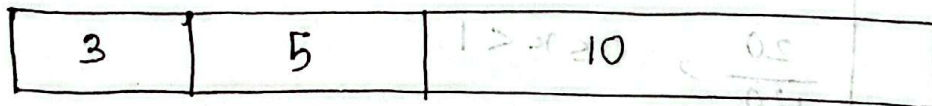
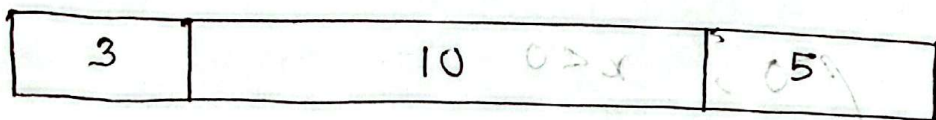
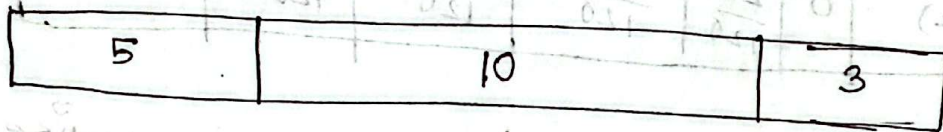
- * criteria (happy path, unhappy path)
- * scenario based $\Rightarrow$ 1st, 2nd, 3rd ques flow
- * maintain $\Rightarrow$

— X —

Greedy Algorithms Ordering Paradigm

Optimal storage on a tape:

$n=3, l=5, 10, 3$



3! ways to put them in a tape.

$$\text{Retrieval time} = \sum_{1 \leq k \leq j} l_k$$

? Expected Retrieval time = $\left(\frac{1}{n}\right) \sum_{1 \leq k \leq j} l_k$

↓
probability

$$l = 5, 3$$

3	5
---	---

$$\frac{3}{1st} + \frac{3+5}{2nd} = 11$$

5	3
---	---

$$\frac{5}{1st} + \frac{5+3}{2nd} = 13$$

retrieval of k^{th} song depends on the summation of length $[1 \text{ to } k-1]$

* sort ascending

$$\{2, 3, 4, 8, 2, 1\} \rightarrow 1$$

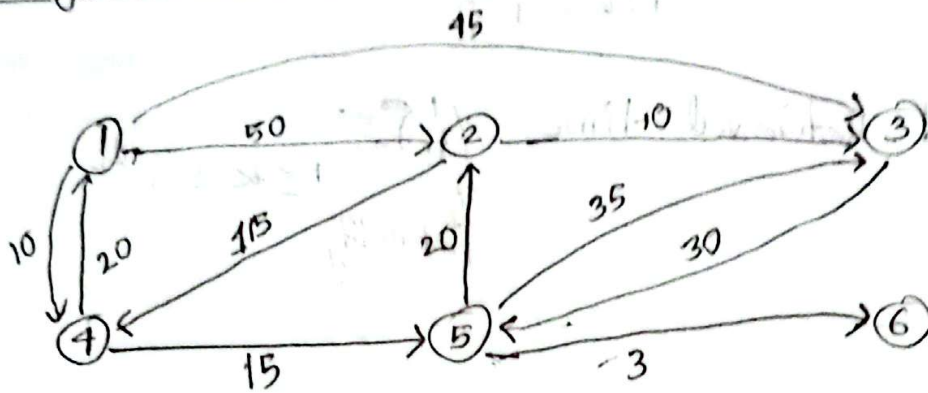
$$\{ \dots, (2, 4), (2, 1), (1, 2), (1, 1) \} \rightarrow 1$$

assume all partitions 2 to n are sorted

$$\{1\} \rightarrow 1$$

$$\{2\} \rightarrow 1$$

single source shortest Path :



In a city there are some junction of roads and from each junction to another junction we are given the travel time / distance. Now there is a source junctions from where we need to find the shortest time / ~~dist~~ distance to all other junction.

Lets define $G(V, E)$, where V is the set of vertices (here it is the junctions), and E is the set of edges with weights (distance / travel time).

$$V = \{1, 2, 3, 4, 5, 6\}$$

$$E = \{(1, 2, 50), (2, 3, 10), (1, 3, 45), \dots\}$$

we have also a set S denoting the sources

$$S = \{1\}$$

$$D = V - S$$

$\text{Dist}(x) = \{1, v_0, v_1, \dots\}$
 ordered $\rightarrow$ such that $\sum_{i, v_0 \dots x} E(i, i+1)$ minimized.

$$\text{dist}[1] = 0; \quad 1 \rightarrow \{2, 3, 4\}$$

$$\text{dist}[4] = \text{dist}[1] + \text{cost}(1, 4) = 10$$

$$\text{dist}[3] = \text{dist}[1] + \text{cost}(1, 3) = 45$$

$$\times \text{dist}[2] = \text{dist}[1] + \text{cost}(1, 2) = 50$$

$$4 \rightarrow \{1, 5\}$$

$$\text{dist}[5] = \text{dist}[4] + \text{cost}(4, 5) = 10 + 15 = 25$$

$$5 \rightarrow \{2, 6, 3\}$$

$$\text{dist}[2] = \text{dist}[5] + \text{cost}(5, 2) = 25 + 20 = 45$$

$$\text{dist}[6] = \text{dist}[5] + \text{cost}(5, 6) = 25 + 3 = 28$$

$$\times \text{dist}[3] = \text{dist}[5] + \text{cost}(5, 3) = 25 + 35 = 60$$

$$6 \rightarrow \{ \}$$

$$2 \rightarrow \{13, 4\}$$

don't need to update cause it's already in it's optimal distance.

$$\text{dist}[3] = \text{dist}[2] + \text{cost}(2, 3) = 45 + 10 = 55$$

$$13 \rightarrow \{5\}$$

Algorithm Single Source Shortest Path, (V, E, S) {

dist[] $\rightarrow$ initialized to ∞

S[] $\rightarrow$ set of optimal distanced node/vertices

dist[S] = 0.0, visit[S] = false

for node: 1 to n do {

if visit[node] true then continue,

visit[node] = true;

for adjNode of node {

for num: 2 to n do {

node = minimum distance node where visit of node is false.

visit[node] = true;

~~for adjNode of~~

for adjNode of node {

if dist[adjNode] > dist[node] + cost(node, adjNode)

{

dist[adjNode] = dist[node] + cost (node, adjNode);

}

}

Parent [adjNode] = node ; $\rightarrow$ to keep track of the path

}

using minheap ~~to the~~ time complexity can be

reduced to $E \log V$.

$$T(n) = T(n-1) + (1-n) \cdot T(n-2)$$

Algorithm

Dynamic Programming

4 steps to solve dp problems.

⇒ Characterize the structure of an optimal solution.

⇒ Recursively define the value of the optimal solution.

⇒ Compute the value of an optimal solution.

⇒ ~~Constant~~ Construct an optimal solution from computed information.

Necessary characteristics:

⇒ Optimal substructure

⇒ Overlapping subproblems.

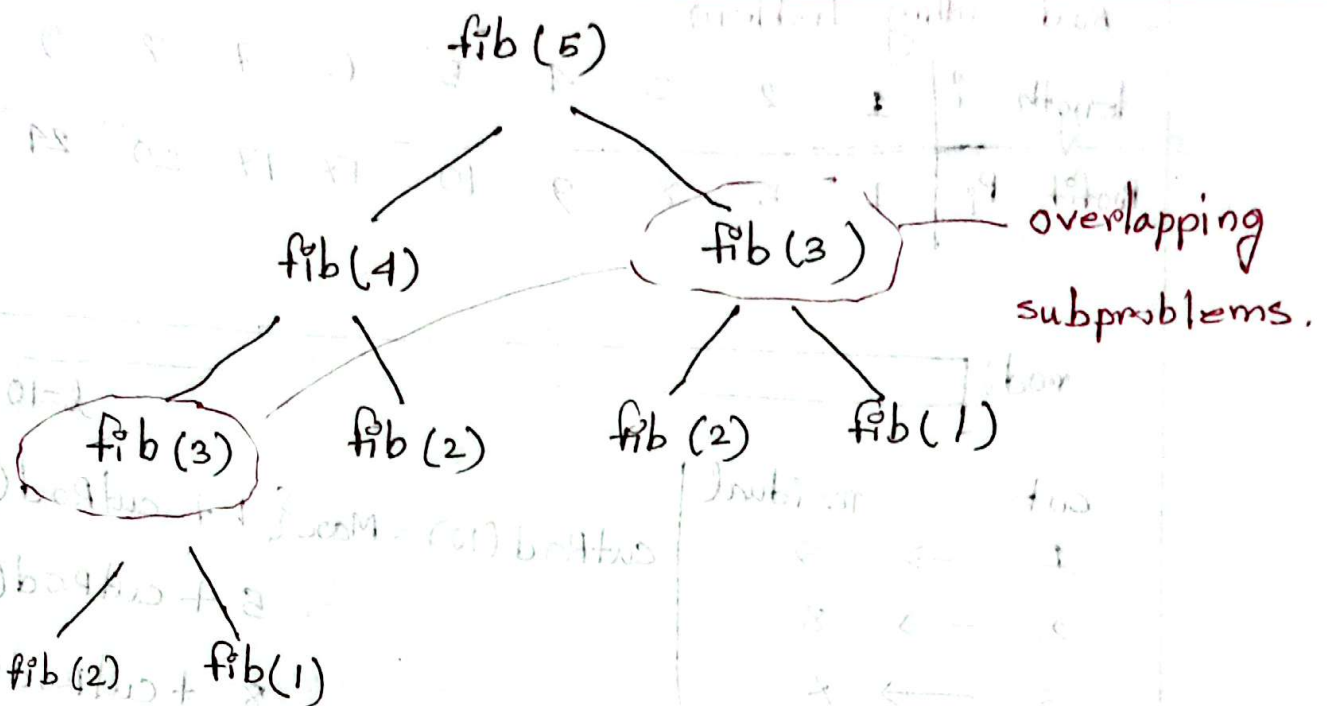
The Fibonacci Problem:

1, 1, 2, 3, 5, 8, 13 - - -

Recursive solution:

$$\text{fib}(n) = \underbrace{\text{fib}(n-1) + \text{fib}(n-2)}_{\text{sub structure}}$$

→ two sub problems.



Redu code

Algorithm $\text{nthFib}(n) \{$

if $(n=1 \text{ or } n=2)$ return 1;

if $(dp[n] \neq -1)$ return $dp[n]$;

int $a := \text{nthFib}(n-1)$;

int $b := \text{nthFib}(n-2)$;

~~return a+b;~~

$dp[n-1] := a$, $dp[n-2] := b$; // Memoization

return $a+b$;

}

* Rod cutting Problem

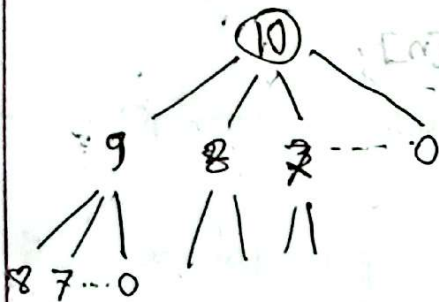
length i	1	2	3	4	5	6	7	8	9	10
Profit P_i	1	5	8	9	10	17	17	20	24	30

rod:

$l=10$

cut	residual
1	9
2	8
3	7
$\vdots$	$\vdots$
10	0

$$\text{cutRod}(10) = \text{Max} \{ 1 + \text{cutRod}(9); \\ 5 + \text{cutRod}(8); \\ 8 + \text{cutRod}(7); \\ \vdots \\ 30 + \text{cutRod}(0); \}$$



```

int cutRod (n) {
    if (n == 0) return 0;
    if (dp[n] != -1) return dp[n];
    int maxProfit = 0;
    for (int i = 1; i <= n; i++) {

```

```

        if (maxProfit < curProfit) {
            maxProfit = curProfit;
            cut[n] = i;
        }
    }
    * to track which cut made the highest profit.

```

```

        int curProfit = P[i] + cutRod(n-i);
        maxProfit = max(maxProfit, curProfit);
    }
    dp[n] = maxProfit;
    return maxProfit;
}

```

$n=10$

P	n
1	9
5	8
8	7
9	6
10	5
17	4
17	3
20	2
24	1
30	0

$n=9$

P	n
1	8
5	7
8	6
9	5
10	4
17	3
17	2
20	1
24	0

$n=8$

P	n
1	7
5	6
8	5
9	4
10	3
17	2
17	1
20	0

$n=7$

P	n
1	6
5	5
8	4
9	3
10	2
17	1
17	0

$n=6$

P	n
1	5
5	4
8	3
9	2
10	1
10	0

$n=4$
 9
 10
 9
 9

$n=4$
 $1 + 3$
 $5 + 2$
 $8 + 1$
 $9 + 0$

$n=3$
 6
 6
 8

$n=3$
 $1 + 2$
 $5 + 1$
 $8 + 0$

$n=2$
 2
 5

$n=2$
 $1 + 1$
 $5 + 0$

$n=1$
 $1 + 0$
 return 1

	1	2	3	4	5	6	7	8	9	10
dp	1	5	8	10	13					
cut	1	2	3	2	2					

$7 + 0$
 $1 + 1$
 $2 + 1$
 $2 + 1$
 $1 + 1$
 $0 + 1$

0
 3
 2
 1
 2
 1
 0

0
 5
 3
 2
 1
 0
 1
 0

$8 + 0$